Práctica 04



Antonio Miguel Almagro Valles
Javier Díaz Vilchez
Rodrigo Iáñez Huertas

Índice

Índice.....**1**
Objeto.....**2**
Antecedentes.....**3**
Estudio de la implementación.....**3**
GreedyBTContLoading.....3
RecursiveBTContLoading1.....4
RecursiveBTContLoading2.....4
RecursiveBTContLoading3.....4
IterativeBTContLoading.....5
Estudio Teórico.....**5**
GreedyBTContLoading.....5
RecursiveBTContLoading1.....6
RecursiveBTContLoading2.....6
RecursiveBTContLoading3.....6
IterativeBTContLoading.....6
Comparativa métodos algorítmicos.....7
Estudio experimental.....**7**
Anexo.....**13**
Fuentes bibliográficas.....**13**

Objeto y Antecedentes

El objetivo de esta práctica es proporcionar un estudio detallado de los algoritmos utilizados para resolver el problema. El problema en cuestión es el de optimizar la carga de un buque portacontenedores. Hay un conjunto de contenedores, que deben ser transportados, cada uno con su peso, en un buque con una capacidad máxima. El objetivo es que estos buques salgan del puerto con la mayor carga posible, respetando su Capacidad Máxima de Carga (CMC). Matemáticamente se puede explicar de la siguiente forma:

Dado un conjunto de contenedores

$$C = \{c_1, c_2, c_3, \ ..., c_n\}$$

, y cada uno con un peso conocido

$$p_i$$

, y un buque con una capacidad máxima de carga (CMC)

$$P$$

, queremos encontrar el subconjunto de contenedores

$$S \subseteq C$$

teniendo en cuenta que la suma de los pesos de los contenedores del subconjunto sea la máxima permitida, teniendo en cuenta que no puede exceder el CMC. Es decir,

$$\sum_{c_i \in S} p_i \leq P$$

Los métodos algorítmicos para resolver este problema son **backtracking** y **branch-and-bound**. A continuación se va a explicar teóricamente en qué consisten ambos métodos:

El método backtracking construye de manera incremental posibles soluciones al problema planteado. Este se hace hasta que, o bien, encuentra un subconjunto de

contenedores que sea igual al CMC, o hasta que se infrinja la restricción. si esto ocurre, el algoritmo vuelve hacia atrás para probar una alternativa diferente. Esto se repite hasta que se obtiene una solución o se recorre todo el conjunto de trabajo.

El método branch-and-bound, es parecido a backtracking, pero este utiliza una función límite para descartar ciertos candidatos sin tener que considerar todas las posibles extensiones de estos subconjuntos candidatos. Si en algún momento, se determina que no es posible obtener una solución a partir de una parcial, se descarta, lo que permite centrarse en las soluciones parciales más prometedoras. Este método es más eficiente ya que no explora todo el espacio de soluciones.

En esta práctica el líder ha sido Antonio Miguel Almagro Valles aunque todos los miembros del grupo han participado de manera activa en la realización de las mismas . Antonio Miguel Almagro Vallesse ha encargado de supervisar cada una de las tareas que hemos realizado, que han sido :

- GreedyBTContLoading → Antonio
- RecursiveBTContLoading1 → Rodrigo
- RecursiveBTContLoading2 → Javier
- RecursiveBTContLoading3 → Javier
- IterativeBTContLoading → Rodrigo
- Main y pruebas → Antonio y Javier
- Tests → Rodrigo
- Estudio de la implementación → Javier
- Estudio teórico → Antonio
- Estudio experimental → Rodrigo

Estas han sido las tareas que se han asignado para cada uno, aunque siempre con la supervisión de los demás miembros del grupo. El trabajo extra que se ha realizado en la práctica pero que no se ve reflejado en la repartición de tareas se ha realizado, en su mayoría, de forma conjunta por los miembros del equipo.

Estudio de la implementación

Para esta práctica, vamos a proporcionar una solución al problema propuesto de carga marítima. El problema es el siguiente: tenemos como datos un buque con cierta capacidad y una serie de contenedores, cada uno con una capacidad también. El problema propuesto es implementar una serie de algoritmos que escojan los contenedores para que la suma de sus pesos sea la máxima posible y la más cercana a la capacidad máxima del buque.

Como punto inicial, hemos creado las clases Buque y Contenedor para identificar cada uno de los atributos de los contenedores y el buque. Por la parte de los contenedores, tienen un método compareTo() en función de sus pesos, al igual que un método equals en el cual 2 contenedores son iguales si tienen los mismos pesos. Además, cada contenedor dispondrá de una ID, entero que servirá para identificarlo y su correspondiente peso.

Por la parte del buque, cada buque dispondrá de un atributo capacidad y una lista de contenedores que no serán los que cargamos, si no los que dispondremos en cada problema para cargar en el buque.

Comenzamos con el primer algoritmo Greedy:

GreedyBTContLoading

La logística para este algoritmo es la siguiente: Ordenar los contenedores en función de su peso y cargar el buque con los de menor peso con el objetivo de cargar un mayor número de contenedores, hasta que se hayan cargado todos los contenedores o no se puedan cargar más en el buque. Como resultado, nos dará una solución más rápida pero no más correcta que los siguientes algoritmos de backtracking, ya que veremos errores en comparación con dichos algoritmos. Su implementación es la siguiente:

Como estructuras se ha optado por una cola de prioridad que ordenaría los contenedores en función del peso. Iteramos sobre
esta cola habiendo guardado previamente todos los contenedores de los que dispone el buque y vamos cargando contenedores hasta que el buque no pueda cargar más o se hayan cargado todos los contenedores.

RecursiveBTContLoading1

La logística para este algoritmo se basa en backtracking, es decir, un algoritmo recursivo que realiza una "vuelta hacia atrás" para comprobar cuál es el mejor resultado de todos los encontrados. Para este primer algoritmo recursivo, el espacio de soluciones se verá reflejado en un árbol binario de soluciones. La implementación es la siguiente:

El método backtrack recibe por parámetro una lista de actualesCargados y mejoresCargados (inicialmente vacía). En cada iteración recursiva del método backtrack, se comprueba previamente si el peso actual en dicha recursión es mayor que el peso máximo para la mejor solución, y si es así, se limpia la lista y se añaden

todos los contenedores actuales a dicha lista, ya que serían la mejor solución. Para cada recursión, se realizan dos llamadas recursivas en el código: una incluyendo al contenedor actual y otra sin incluirlo y se verifica con la condición anterior si existe mejora, hasta que se encuentre la mejor solución.

RecursiveBTContLoading2

Para este algoritmo, su función es similar a la del recursivo 1. Esta vez, la condición que comparamos es si el peso actual + el peso del contenedor actual es menor o igual que la capacidad del buque, lo que nos asegura que podemos continuar en la iteración y llamamos recursivamente incluyendo el contenedor actual y excluyéndose. Esto nos asegura no considerar los subárboles situados a la derecha, ya que no contienen mejores soluciones que los situados a la izquierda. Las soluciones las guardaremos en un treemap con clave el peso y valor una lista con los contenedores cargados y la mejor solución será la entrada con mayor clave o peso.

RecursiveBTContLoading3

Para este algoritmo, añadimos 2 parámetros adicionales: un array de enteros con la solución actual de los contenedores cargados y otro array de enteros soa con la solución óptima actual. Las condiciones son las mismas, solo que ahora el funcionamiento para cargar una mejor solución actual en la óptima es la siguiente: ambos arrays de enteros solo tendrán valores de 0 y 1, 1 si el contenedor es elegido y 0 si no lo es. Si el peso actual es mayor que el peso máximo encontrado en todas las soluciones, el array de enteros s con la solución actual se copia en el array de la mejor solución óptima encontrada entre todas las soluciones. De esta forma, al terminar la recursividad, nos quedará un array de enteros con 1 y 0, donde los 1 serán los contenedores elegidos.

IterativeBTContLoading

Con este algoritmo, se añade una mejora más al anterior eliminando el espacio de la pila de recursividad. Utiliza un bucle do-while para explorar todas las combinaciones posibles de contenedores. En cada iteración: Incrementa el nivel para probar si el contenedor actual se incluye en la solución o no. Si s[nivel] es 1, significa que el contenedor actual se incluye, por lo que se incrementa pact con el peso del contenedor en el nivel actual. Si s[nivel] es -1, significa que el contenedor no se

incluye, y se decrementa pesoRestante con el peso del contenedor en el nivel actual. Si se alcanza el último nivel, verifica si la combinación actual (pact) es mejor que la mejor encontrada hasta ahora y no excede la capacidad. Si es así, actualiza voa y la lista de la mejor solución soa con los contenedores correspondientes.

Si no se ha alcanzado el último nivel: Verifica si la combinación actual puede llevar a una mejor solución futura. Si es así, incrementa el nivel para explorar más contenedores. Si no, retrocede al nivel anterior ajustando pact y pesoRestante según corresponda. El bucle continúa hasta que se hayan explorado todas las combinaciones posibles.

Estudio Teórico

GreedyBTContLoading

Este algoritmo consta de un bucle while que se repite hasta **n** contenedores, para cada contenedor se hace una operación **poll()**, que es de orden $O(\ log(n)\ )$, por lo que el orden de complejidad temporal de este algoritmo es $O(\ n\ \times\ log(n)\ )$.

La complejidad espacial es tenemos que fijarnos en el espacio en memoria que ocupa, en este caso se inicializan varias variables al principio, pero la más significativa es una cola de prioridad (**PriorityQueue<Contenedor>**) de tamaño **n**, siendo n el número de contenedores. Por lo que el orden espacial de este algoritmo es $O(n\ )$.

RecursiveBTContLoading1

El siguiente algoritmo, y esto se puede aplicar a todos los algoritmos de backtracking, es de $O(\ 2^n\ )$. Para cada contenedor, hay dos opciones, incluirlo en el buque, o no. Por lo tanto, si tenemos **n** contenedores, tendremos 2^n opciones opciones posibles que explorar. Para cada contenedor se duplica el número de posibles combinaciones.

Para calcular el orden espacial del algoritmo, hay que fijarse en la pila de llamadas recursivas, en este caso en la pila puede haber hasta **n** contenedores en el peor de los casos. Por lo que el orden espacial será $O(\ n\ )$.

RecursiveBTContLoading2

El orden temporal es el mismo que en el **`RecursiveBTContLoading2`**, ya que de igual forma para cada contenedor hay dos posibles oposiciones, por lo que es $O(2^n)$.

En este caso, el algoritmo usa, además de la pila, una estructura de datos TreeMap para almacenar todas las opciones encontradas. En el peor de los casos, esta sería de tamaño 2^n. Por el orden espacial de este algoritmo sería $O(n \times 2^n)$.

RecursiveBTContLoading3

El orden de complejidad temporal de este algoritmo es de la misma forma que los anteriores ya que sigue la misma lógica. Es de $O(2^n)$**.**

Con respecto a su complejidad espacial, la diferencia con el algoritmo anterior, es que este algoritmo utiliza dos matrices para rastrear la solución actual, y otra para almacenar la solución final. Ambas tienen un tamaño n, por lo que el orden de complejidad espacial es $O(n)$.

IterativeBTContLoading

De igual forma que en los algoritmos anteriores, al tratarse de un método basado en backtracking y que para cada contenedor haya dos posibles opciones, el orden de complejidad de este algoritmo es $O(2^n)$**.**

Si nos fijamos en su complejidad espacial, este algoritmo tiene un array donde se almacena el número de contenedores, y una lista que en el pero de los casos contiene todos los contenedores, por lo que sería de tamaño n. Por lo tanto, el orden de complejidad espacial del algoritmo es $O(n)$.

Comparativa métodos algorítmicos

Técnica algorítmica	Ventajas	Desventajas
Greedy	- Rápidos. - Fáciles de implementar. - Recomendables si la solución óptima local conduce a la solución óptima global.	- No siempre encuentran la solución óptima global. - No son buenos para problemas con restricciones. - Dependen de la elección inicial

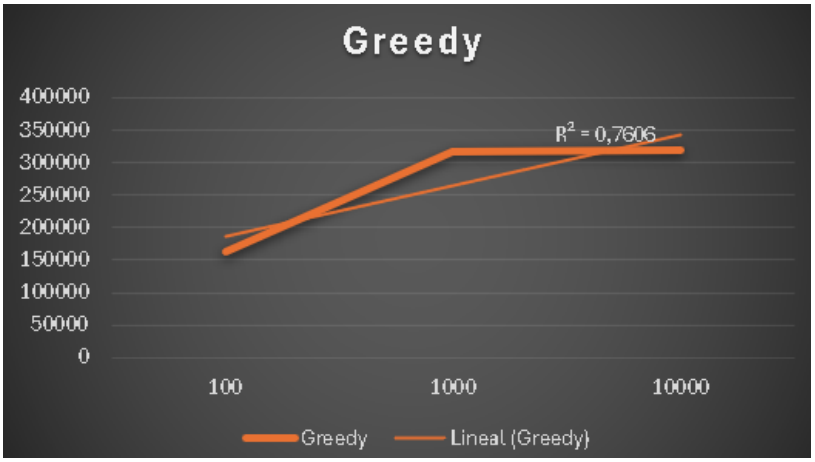
Backtracking	- Pueden encontrar todas las soluciones posibles. - Garantizan la solución óptima. - Efectivos para resolver problemas con un gran espacio de soluciones.	- Ineficientes si el espacio de soluciones es amplio. - Las restricciones no permiten eliminar muchas soluciones.
Branch-and-Bound	- Garantizan la solución óptima. - Son más eficientes que backtracking.	- Complejos de implementar. - Pueden requerir más memoria para almacenar los subproblemas.

Estudio experimental

Para la primera prueba, hemos mantenido los mismos contenedores y variado la capacidad del buque, obteniendo los siguientes resultado para un buque de capacidad 100, 1000 y 10000, los contenedores han sido los siguientes:

Ingrese el peso del contenedor 1: 50
Ingrese el peso del contenedor 2: 30
Ingrese el peso del contenedor 3: 20
Ingrese el peso del contenedor 4: 200
Ingrese el peso del contenedor 5: 500
Ingrese el peso del contenedor 6: 750
Ingrese el peso del contenedor 7: 150
Ingrese el peso del contenedor 8: 1000
Ingrese el peso del contenedor 9: 5000
Ingrese el peso del contenedor 10: 8000

Greedy



Recursivo 1



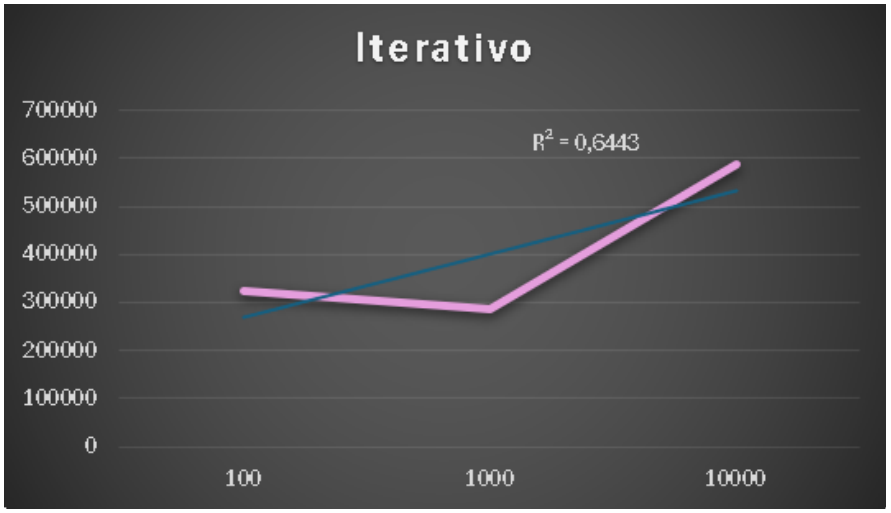
Recursivo 2



Recursivo 3



Iterativo



Para la capacidad del buque 100 el resultado óptimo de todos los algoritmos fue los contenedores 1,2 y 3.

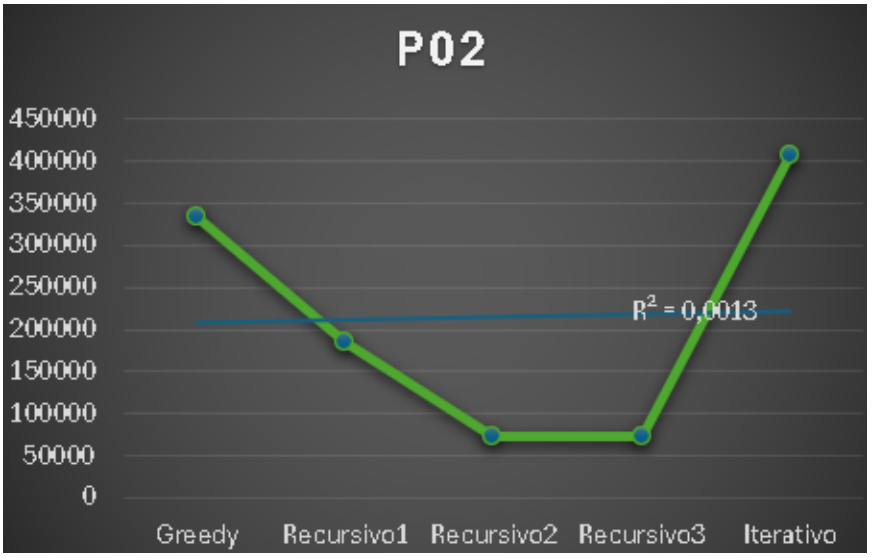
Con una capacidad de 1000, el resultado cambió, para el algoritmo Greedy son los contenedores 3,2,1,7,4 y 5, para el resto de algoritmos fue el contenedor 8.

Por último, para una capacidad de 10000, todos los algoritmos excepto el Greedy escogieron los contenedores 2,3,4,6,8 y 10, para el Greedy fueron 3,2,1,7,4,5,6,8 y 9

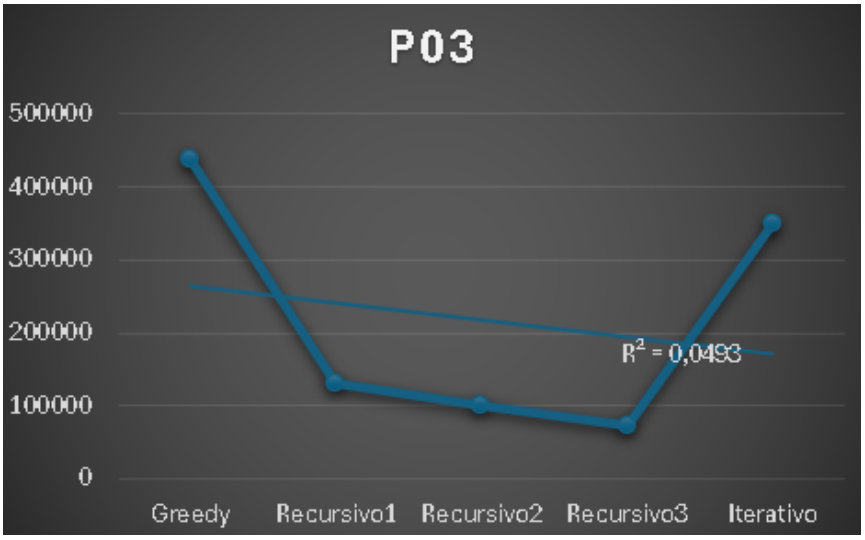
Para la siguiente prueba que realizamos, tuvimos en cuenta los archivos P01, P02, P03 y P04 que se nos proporcionaron, los tres primeros tenían la capacidad del buque a 100, pero cambiando el número y capacidad de los contenedores, obtuvimos los siguientes resultados:



Aquí teníamos 9 contenedores de distinto peso.

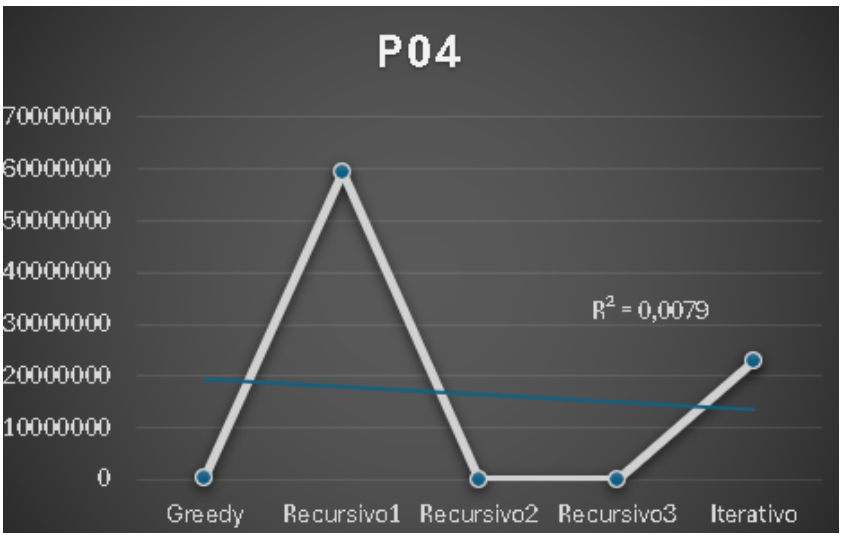


Teniendo en cuenta que habían 14 contenedores de distinto peso.



Y aquí 10 contenedores de distintos peso.

Para el último, se tenía una capacidad de buque de 524 y 33 contenedores con distinto peso:



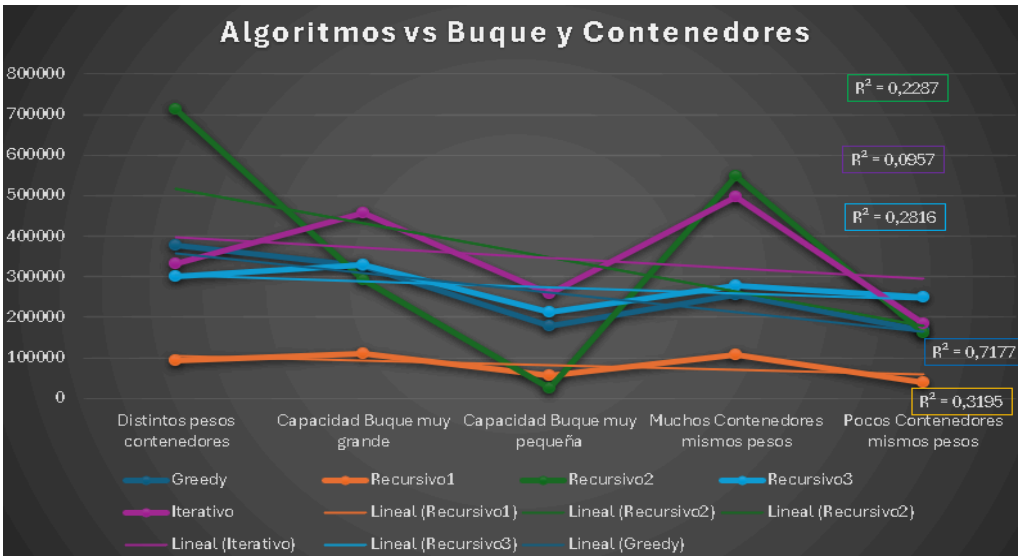
Las últimas pruebas que hemos realizado han sido variables de las que ya hemos realizado anteriormente, creando una tabla con los contenedores que escoge cada algoritmo, y a la izquierda los datos que se han introducido para dichas pruebas. Dándonos unos resultados distintos para cada algoritmo, como se puede observar a continuación:

	Greedy	Recursivo1	Recursivo2	Recursivo3	Iterativo
Distintos pesos contenedores	7,5,6,8,4,1,3	2,3,4,5,8	2,3,4,5,8	2,3,4,5,8	2,3,4,5,8
Ingrese la capacidad del buque: 1000					
Ingrese el número de contenedores: 8					
Ingrese el peso del contenedor 1: 200					
Ingrese el peso del contenedor 2: 400					
Ingrese el peso del contenedor 3: 300					
Ingrese el peso del contenedor 4: 150					
Ingrese el peso del contenedor 5: 50					
Ingrese el peso del contenedor 6: 85					
Ingrese el peso del contenedor 7: 40					
Ingrese el peso del contenedor 8: 100					
Capacidad Buque muy grande	1,2,3,4,7,5,6,10	9 y 10	9 y 10	9 y 10	9 y 10
Ingrese la capacidad del buque: 100000					
Ingrese el número de contenedores: 10					
Ingrese el peso del contenedor 1: 500					
Ingrese el peso del contenedor 2: 1000					
Ingrese el peso del contenedor 3: 1500					
Ingrese el peso del contenedor 4: 3500					
Ingrese el peso del contenedor 5: 4000					
Ingrese el peso del contenedor 6: 5500					
Ingrese el peso del contenedor 7: 3500					
Ingrese el peso del contenedor 8: 50000					
Ingrese el peso del contenedor 9: 60000					
Ingrese el peso del contenedor 10: 40000					

Capacidad Buque muy pequeña	1,2,3,4,7	9 y 10	9 y 10	9 y 10	9 y 10
Ingrese la capacidad del buque: 100					
Ingrese el número de contenedores: 10					
Ingrese el peso del contenedor 1: 5					
Ingrese el peso del contenedor 2: 10					
Ingrese el peso del contenedor 3: 15					
Ingrese el peso del contenedor 4: 35					
Ingrese el peso del contenedor 5: 40					
Ingrese el peso del contenedor 6: 55					
Ingrese el peso del contenedor 7: 35					
Ingrese el peso del contenedor 8: 50					
Ingrese el peso del contenedor 9: 60					
Ingrese el peso del contenedor 10: 40					
Muchos Contenedores mismos pesos	1,10,9,8,7...	todos	todos	todos	todos
Ingrese la capacidad del buque: 10000					
Ingrese el número de contenedores: 10					
Ingrese el peso del contenedor 1: 60					
Ingrese el peso del contenedor 2: 60					
Ingrese el peso del contenedor 3: 60					
Ingrese el peso del contenedor 4: 60					
Ingrese el peso del contenedor 5: 60					
Ingrese el peso del contenedor 6: 60					
Ingrese el peso del contenedor 7: 60					
Ingrese el peso del contenedor 8: 60					
Ingrese el peso del contenedor 9: 60					
Ingrese el peso del contenedor 10: 60					

Pocos Contenedores mismos pesos	1,5,4,3...	todos	todos	todos	todos
Ingrese la capacidad del buque: 1000					
Ingrese el número de contenedores: 5					
Ingrese el peso del contenedor 1: 180					
Ingrese el peso del contenedor 2: 180					
Ingrese el peso del contenedor 3: 180					
Ingrese el peso del contenedor 4: 180					
Ingrese el peso del contenedor 5: 180					

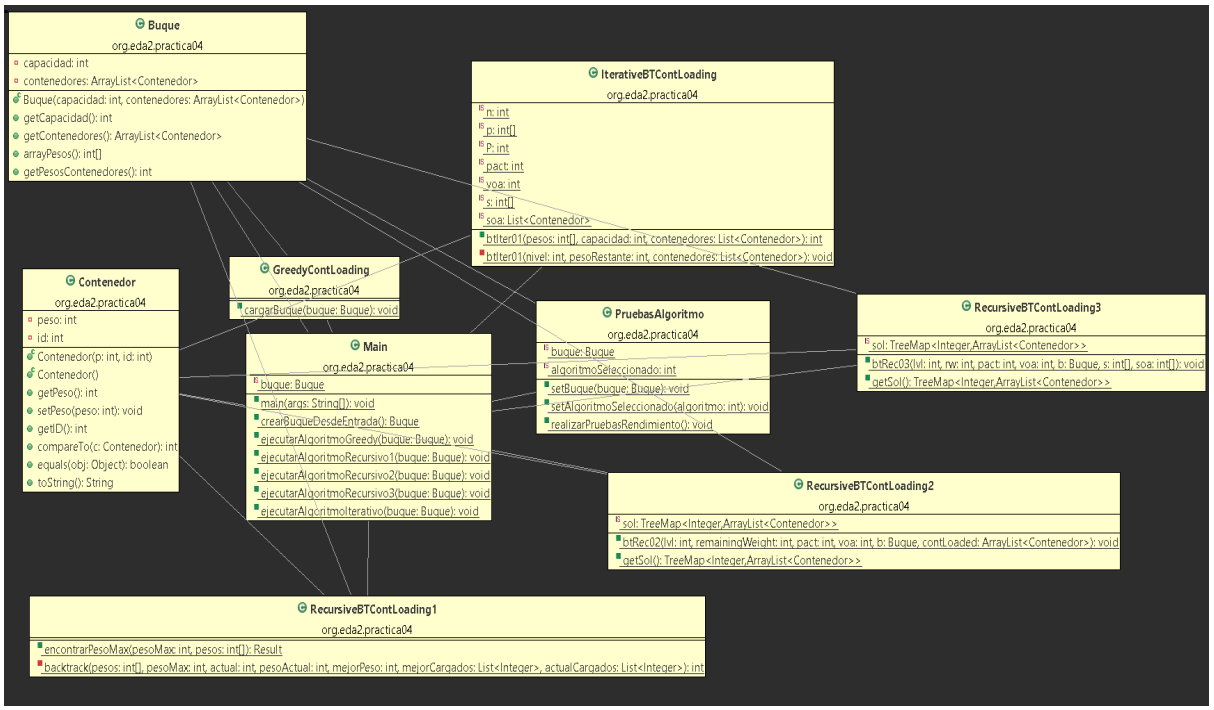
Los resultados obtenidos para cada algoritmo se muestra en la siguiente gráficas:



Todos los datos obtenidos se han calculado en nanosegundos, ya que hemos pensado que sería más fiable a la hora de obtener las gráficas y mucho más precisos. Como se puede observar, R^2 la mayoría no superan el 0.95, por lo que la toma de datos no es correcta y no podemos fiarnos de los tiempos obtenidos, por lo que las gráficas podrían variar.

Anexo

El Diagrama de Clases obtenido es:



Donde el Main, Buque y Contenedores tienen relación con el resto, ya que son necesarios para la ejecución del código.

Conclusión

En conclusión, la práctica ha demostrado que el uso del backtracking es eficiente para resolver el problema de la asignación de contenedores en buques. Esta técnica, que explora sistemáticamente todas las posibles configuraciones, ha permitido encontrar soluciones al problema. Sin embargo, basándonos en nuestros conocimientos teóricos y la naturaleza del problema, podemos deducir que un algoritmo basado en branch-and-bound podría ser más eficiente.

El branch-and-bound reduce el espacio de búsqueda utilizando estimaciones para descartar ciertas ramas que no conducirán a la solución óptima, lo que resulta en una reducción significativa del tiempo de cálculo. Aunque esta técnica requiere una formulación más compleja, su capacidad para eliminar subconjuntos de soluciones no válidas lo convierte en un candidato superior para este tipo de problemas de optimización.

Por lo tanto, aunque el backtracking ha sido útil, se recomienda hacer uso de algoritmos de branch-and-bound para mejorar la eficiencia en la resolución del problema.

Fuentes bibliográficas

https://people.sc.fsu.edu/~jburkardt/datasets/bin_packing/bin_packing.html
https://www.youtube.com/watch?v=DKCbsiDBN6c
https://algorithm-visualizer.org/backtracking/hamiltonean-cycles